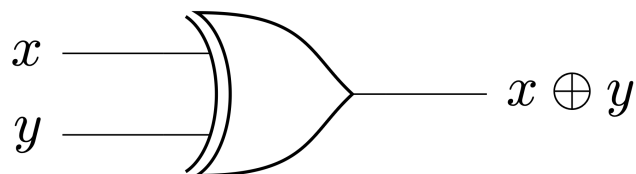


The **exclusive-OR** of x and y , written $x \oplus y$, is defined as

$$x \oplus y = xy' + x'y.$$

The output is 1 when exactly one of the two inputs is 1 — that is, when the inputs differ. It is 0 when the inputs are the same (both 0 or both 1).

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

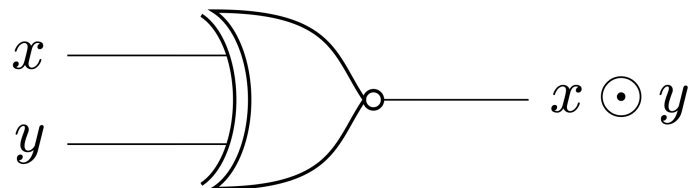


The **exclusive-NOR** is the complement of XOR, written $x \odot y$. It is defined as

$$x \odot y = xy + x'y'.$$

It is 1 when the two inputs are the same (both 1 or both 0). This is why it is also called **equivalence**, it checks whether the inputs are equal.

x	y	$x \odot y$
0	0	1
0	1	0
1	0	0
1	1	1



We have some useful identities for XOR. These come up again and again. Each can be checked with a quick truth table.

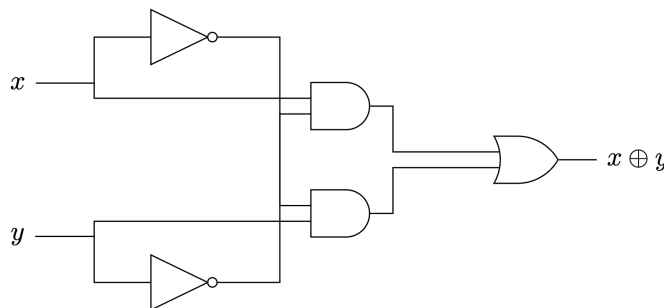
Identity	Why it makes sense
$x \oplus 0 = x$	differing from 0 leaves x unchanged
$x \oplus 1 = \bar{x}$	differing from 1 flips x
$x \oplus x = 0$	a value never differs from itself
$x \oplus \bar{x} = 1$	a value always differs from its complement
$x \oplus \bar{y} = \bar{x} \oplus y = \overline{x \oplus y}$	complementing one input flips the result

XOR is both commutative and associative:

$$A \oplus B = B \oplus A \quad (A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$$

Because order and grouping don't matter, we can write three or more XORs with no parentheses. So in principle we could imagine a 3-input or 4-input XOR gate.

A 2-input XOR can be made from two inverters, two AND gates, and one OR gate, directly mirroring $xy' + xy'$. When you look at a XOR logic gate, this is actually what is happening on the inside:



Example 1 — Simplify $a \oplus a \oplus b$

Work left to right. We know $a \oplus a = 0$, so

$$a \oplus a \oplus b = (a \oplus a) \oplus b = 0 \oplus b = b.$$

Example 2 — Show that $x \oplus 1 = \bar{x}$

Use the definition with $y = 1$:

$$x \oplus 1 = x \cdot \bar{1} + \bar{x} \cdot 1 = x \cdot 0 + \bar{x} \cdot 1 = \bar{x}.$$

Example 4 — Evaluate a 4-input XOR

Compute $1 \oplus 0 \oplus 1 \oplus 1$. Because XOR is associative, just count the 1's: there are three 1's, an *odd* number, so the result is 1.

$$1 \oplus 0 \oplus 1 \oplus 1 = 1.$$

Example 4 gives us an important insight into easily computing a XOR function.

- **When there is an ODD number of 1s in the XOR expression, the output is always 1.**
- **When there is an EVEN number of 1s in the XOR expression, the output is always 0.**

XNOR (equivalence) gives us an instant equality test. Since $x \odot y = 1$ exactly when x and y are equal, a single XNOR gate is a 1-bit comparator. Computers cannot determine whether two binary numbers are equal just by looking at them like we do, so they use XNOR gates to determine this.

Example 6 — Comparing two 2-bit numbers

Suppose we want to check whether two 2-bit numbers $A = A_1A_0$ and $B = B_1B_0$ are equal. They are equal only if *both* bit positions match:

$$\text{Equal} = (A_1 \odot B_1) \cdot (A_0 \odot B_0).$$

Each XNOR checks one bit position; the AND requires both to agree. For example, with $A = 10$ and $B = 10$:

$$(1 \odot 1) \cdot (0 \odot 0) = 1 \cdot 1 = 1 \quad \Rightarrow \quad \text{equal.}$$

With $A = 10$ and $B = 11$:

$$(1 \odot 1) \cdot (0 \odot 1) = 1 \cdot 0 = 0 \quad \Rightarrow \quad \text{not equal.}$$

When we extend XOR to three or more variables, a pattern appears. Convert the 3-variable case to a normal Boolean expression:

$$A \oplus B \oplus C = (\overline{A}\overline{B} + \overline{A}B)\overline{C} + (AB + \overline{A}\overline{B})C = \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}C + ABC.$$

In minterm form: $A \oplus B \oplus C = \sum(1, 2, 4, 7)$.

Look at those minterm numbers in binary: 001, 010, 100, 111. Each one has an odd number of 1's. That is the whole pattern.

A multi-variable XOR is called an **odd function**. It equals 1 exactly when an odd number of its input variables are 1. For n variables, there are a total of $\frac{2^n}{2}$ rows whose input combinations have an odd number of 1s.

Example 9 — Find the minterms of a 3-variable odd function

List the input combinations where $A \oplus B \oplus C = 1$ by counting 1's in each row.

A	B	C	# of 1's	$A \oplus B \oplus C$
0	0	0	0 (even)	0
0	0	1	1 (odd)	1
0	1	0	1 (odd)	1
0	1	1	2 (even)	0
1	0	0	1 (odd)	1
1	0	1	2 (even)	0
1	1	0	2 (even)	0
1	1	1	3 (odd)	1

When we send data over a wire, a bit can unintentionally flip. XOR is good for this type of error detection. A parity bit is a cheap way to catch that.

Parity bit: one extra bit added to the right-end of a message so that the total number of 1's becomes either always even (even parity) or always odd (odd parity).

- A **parity generator** computes the parity bit before sending.
- A **parity checker** verifies parity after receiving.

Example — 3-bit even-parity generator. Message bits are x, y, z . We need parity bit P so the total count of 1's among x, y, z, P is even.

x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Notice $P = 1$ exactly on the rows with an odd number of 1's among x, y, z — that is the odd function again. So

$$P = x \oplus y \oplus z.$$

The checker. The receiver gets four bits x, y, z, P . With even parity, a correct message has an even number of 1's. Define the check output C :

$$C = x \oplus y \oplus z \oplus P.$$

$C = 1$ means an odd number of 1's arrived, which signals an **error** (some bit flipped). $C = 0$ means parity checks out.

Example 7 — Generate the parity bit for a message

The 3-bit message $xyz = 101$ is sent with *even* parity. What is P ?

$$P = x \oplus y \oplus z = 1 \oplus 0 \oplus 1.$$

Count the 1's in the message: two 1's, an even number. To keep the total even, P must be 0:

$$P = 1 \oplus 0 \oplus 1 = 0.$$

So the transmitted word is $xyzP = 1010$, which has two 1's — even, as required. **Lesson:** even parity sets P so the *whole word* has an even number of 1's. If the message already has an even count, $P = 0$; if odd, $P = 1$.